Post Quantum Cryptography on Android

Mario Bodemann

yubico

Post Quantum Cryptography - Dangers for Classics

Dangers of classical computing

- “harvest now, decrypt later”
  - We don’t need to break encryption “now”
  - “Soon” is good enough
  - Let us capture all data we can, for later.
- Shor’s Algorithm
  - Factorization (base for classic cryptography) becomes “easy” with quantum computers
  - Quantum computers with enough qbits arrive “soon”

LLMs: Additional Risk for Classics PKI

- Million Dollar Challenge
  - “Navier-Stokes” on fluid dynamics
  - solved (?) by LLM controversy
  - Expensive token usage
- NIST HAWK
  - LLM found flaws
  - NIST revoked HAWK
- Marketing or Real?
  - Can we risk it

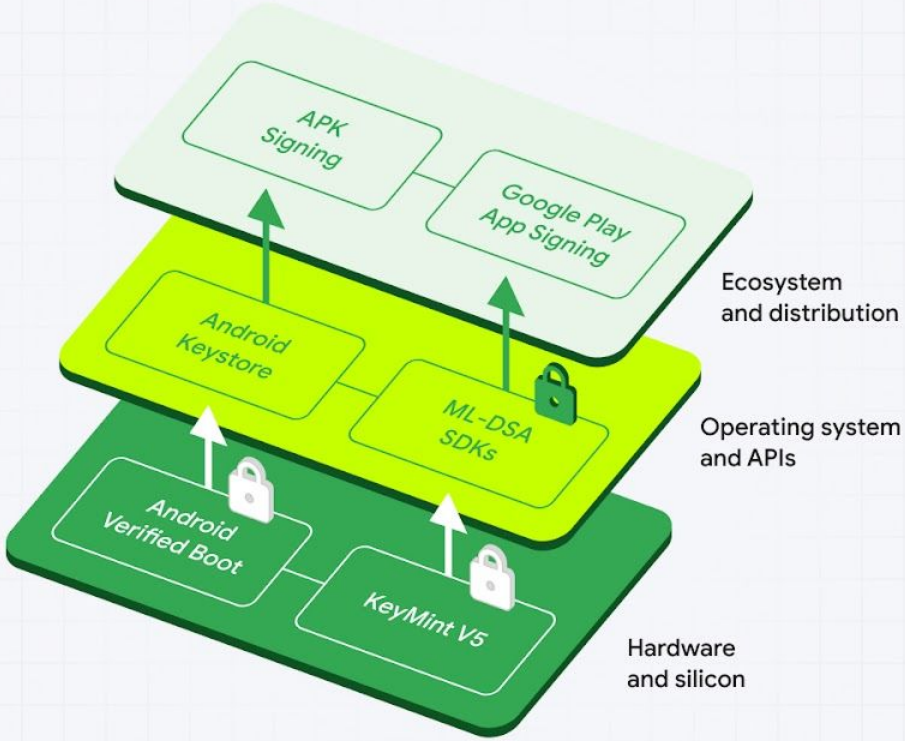
Post Quantum Cryptography - Solutions

- Bigger Key Sizes
- NIST: ML
  - Module-Lattice-Based
    - Key-Encapsulation Mechanism Standard (ML-KEM)
    - Digital Signing Algorithm Standard (ML-DSA)
  - Algorithm with classical means
  - But hard for Quantum Computers to break into
- Key sizes explosion
  - Public key: 32B (EC) to 1.312B (ML-DSA)
  - Private key: 64B (EC) to 2.560B (ML-DSA)
  - Signature: 65B (EC) to 2.420B (ML-DSA)

https://en.wikipedia.org/wiki/Post-quantum_cryptography#Comparison

PQC today

Android 17 Post-Quantum Chain of Trust



The full-stack vision illustrates how Android 17 secures the entire ecosystem, establishing a continuous chain of mathematical certainty from the physical hardware boot sequence through to the global distribution of applications via Google Play.

https://blog.google/security/security-for-the-quantum-era-implementing-post-quantum-cryptography-in-android/

Testing: Sign own APK

- Two plus one certificates
  - old / current certificate from your old app version or google’s keystore
  - New hybrid classic certificate
    - For older phones not understanding pqc
  - New hybrid pqc signature (future proofing)
    - ML-DSA
- 2x certificate lineages
  - Rotate keys from old to new
  - 1x from current classic signature to new hybrid classic signature
  - 1x from current classic signature to new hybrid pqc signature
- Sign with apksigner tool

Testing: Check APK signatures

- Walk through all installed packages
- Check certificates

⚠ Non Android 17 phones

- cannot determine version 3 certificates
- I.e. cannot “see” pqc signatures

or

```
apksigner verify --print-certs-pem --verbose app-release.apk
…
V3.2 Hybrid PQC Signer: {…} key algorithm: ML-DSA
```

Testing: Public Key Infrastructure

```
val kpg = KeyPairGenerator.getInstance(
   algo,
   "AndroidKeyStore"
)
```

- Create a key pair generator
- Use the build in AndroidKeyStore
- Using the algorithm to be tested
  - "ML-DSA"
  - "ML-DSA-44"
  - "ML-DSA-65"
  - "ML-DSA-87"

Testing: Public Key Infrastructure

```
val kpg = KeyPairGenerator.getInstance(
   algo,
   "AndroidKeyStore"
)

val spec = KeyGenParameterSpec.Builder(
   "alias_${algo.lowercase()}",
   KeyProperties.PURPOSE_SIGN or
KeyProperties.PURPOSE_VERIFY
)
   .setIsStrongBoxBacked(useStrongbox)
   .build()
```

- Build a key spec for signing and verifying
- Try to make it run on the strongbox (dedicated hardware)
- Failures get thrown, be ready to catch

Testing: Public Key Infrastructure

```
val kpg = KeyPairGenerator.getInstance(
   algo,
   "AndroidKeyStore"
)

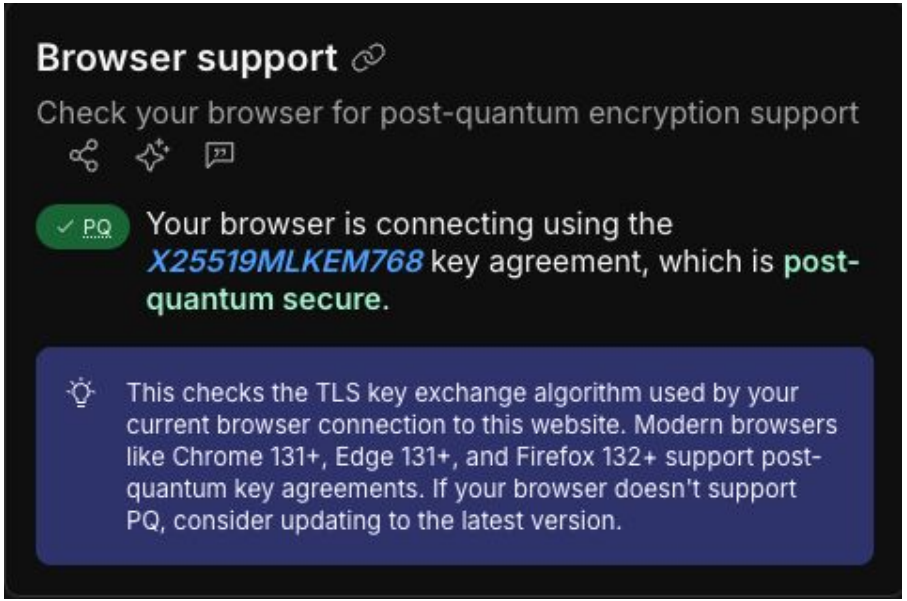
val spec = KeyGenParameterSpec.Builder(
   "alias_${algo.lowercase()}",
   KeyProperties.PURPOSE_SIGN or
KeyProperties.PURPOSE_VERIFY
)
   .setIsStrongBoxBacked(useStrongbox)
   .build()

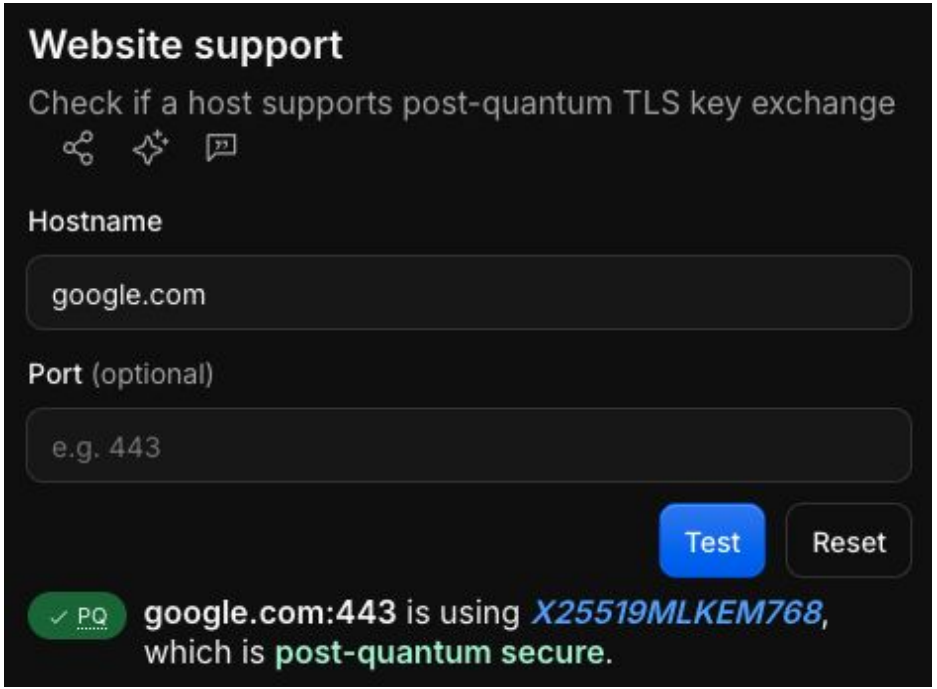
kpg.initialize(spec)
val kp = kpg.generateKeyPair()
val signer = Signature.getInstance(algo)
signer.initSign(kp.private)
signer.update(plainText.toByteArray())

val signature = signer.sign()
```

- Combine specification with generator
- Create a signer (and verifier later) for current algorithm
- Initialize signer with private key and
- Fill it with the plain text
- Finally create the signature

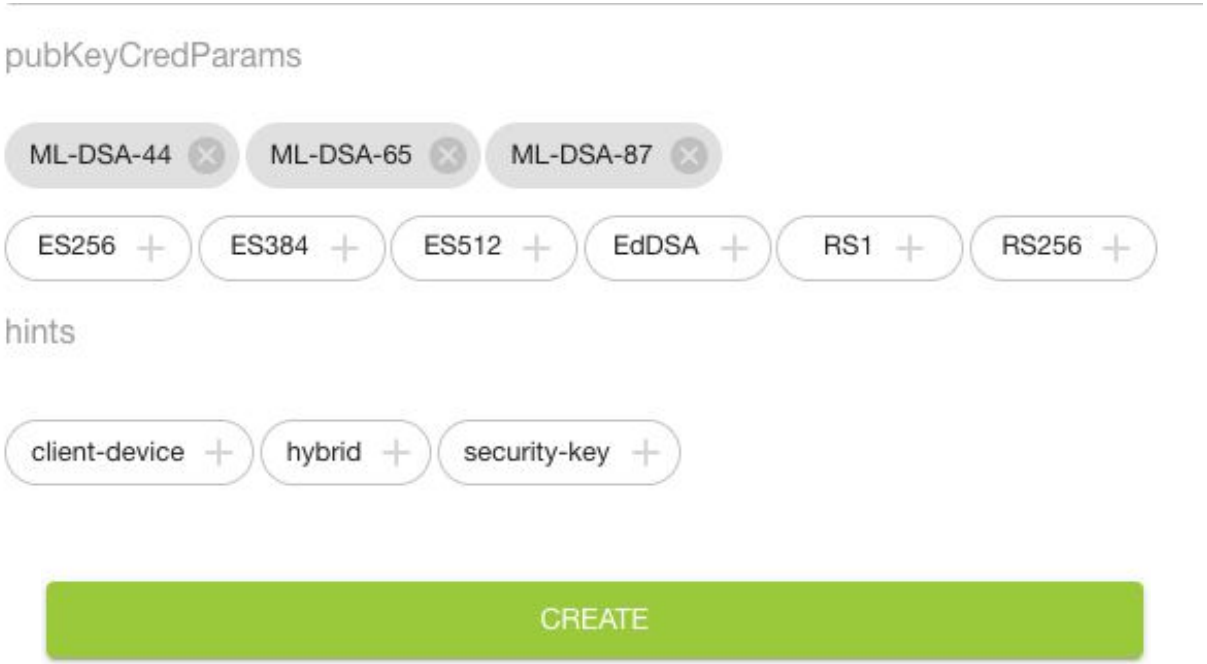
Testing: TLS





https://radar.cloudflare.com/post-quantum

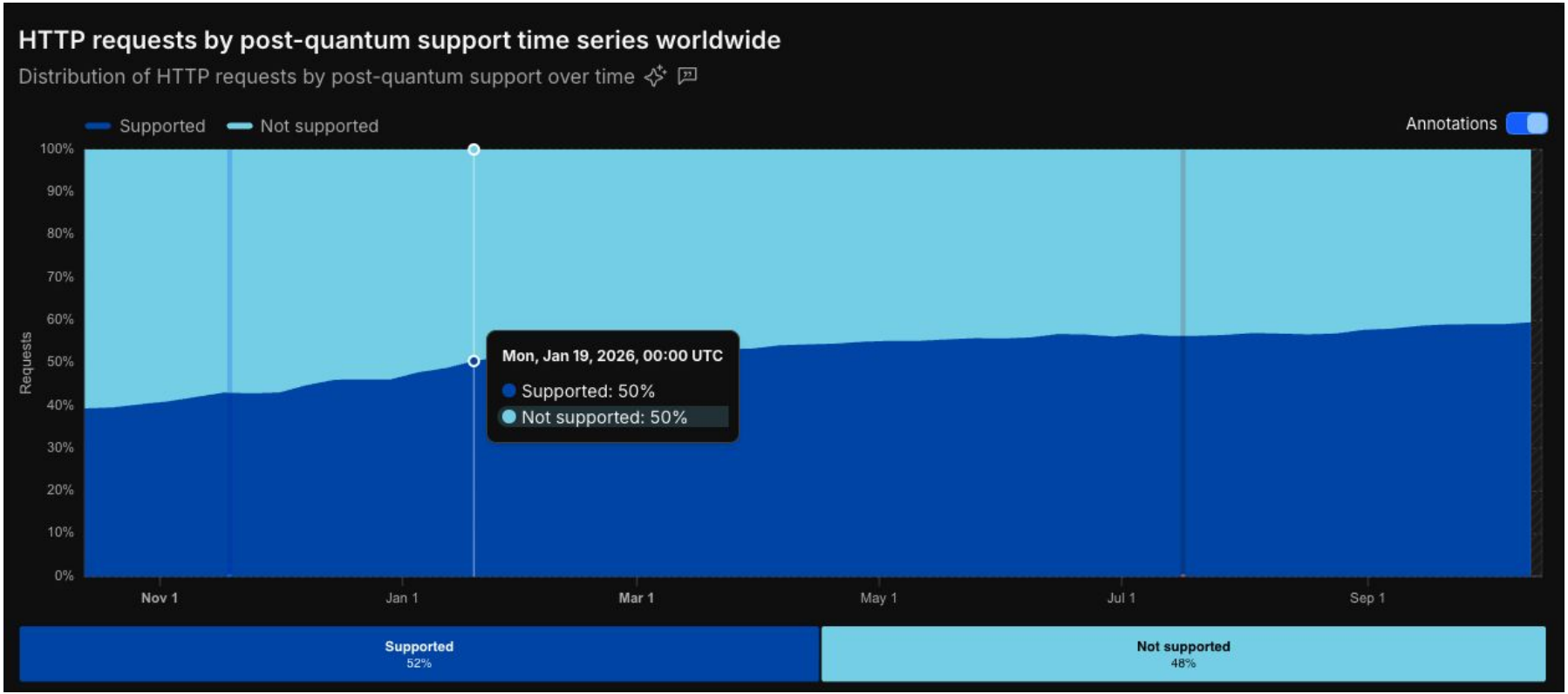
Testing: Credentials (Passkeys)



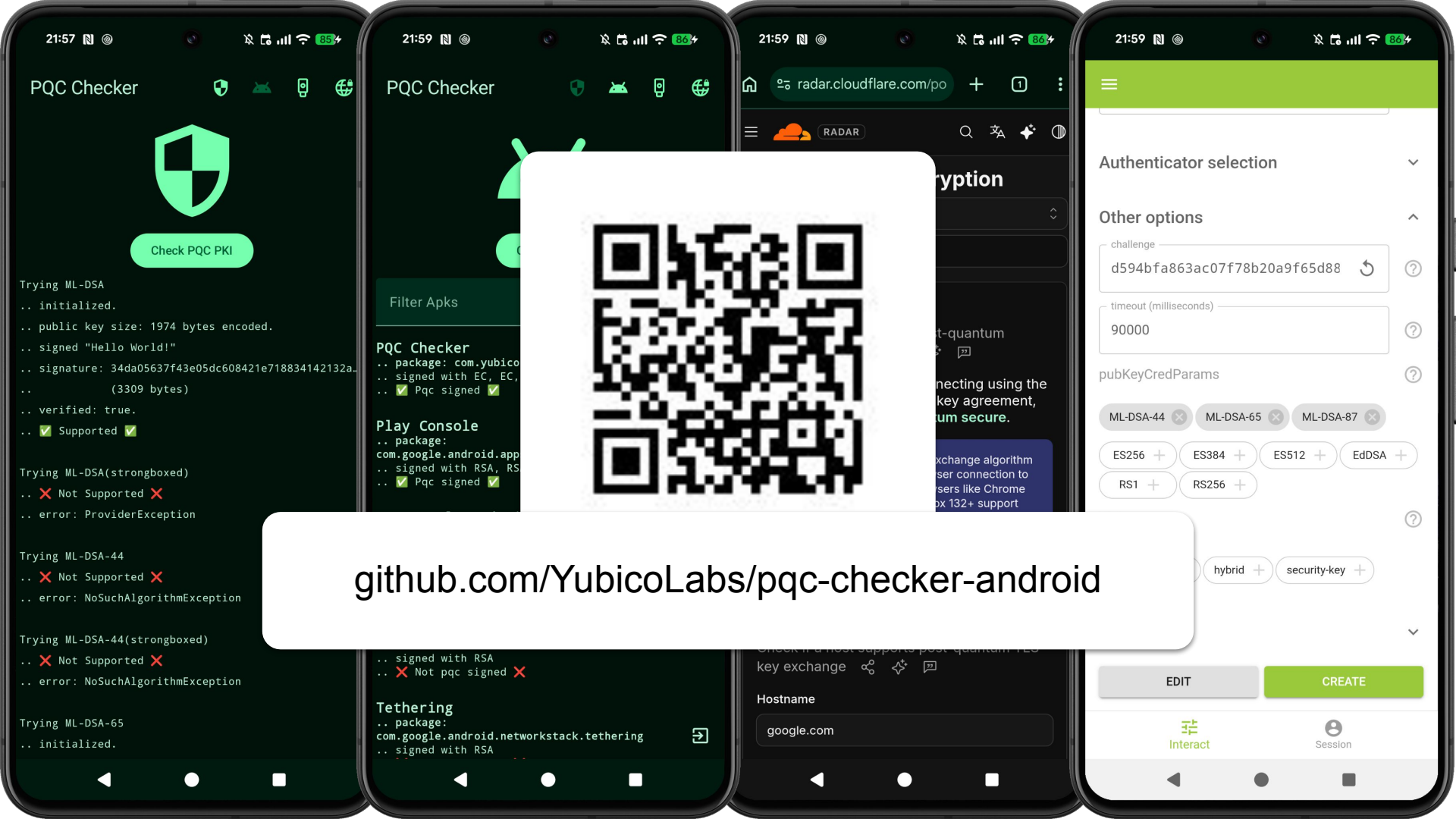
https://demo.yubico.com/webauthn-developers

Summary

Summary



radar.cloudflare.com/explorer?groupBy=post_quantum&dt=52w


PQC Checker
Check PQC PKI
Trying ML-DSA
.. initialized.
.. public key size: 1974 bytes encoded.
.. signed "Hello World!"
.. signature: 34da05637f43e05dc608421e718834142132a..
.. (3309 bytes)
.. verified: true.
.. ✅ Supported ✅
Trying ML-DSA(strongboxed)
.. ❌ Not Supported ❌
.. error: ProviderException
Trying ML-DSA-44
.. ❌ Not Supported ❌
.. error: NoSuchAlgorithmException
Trying ML-DSA-44(strongboxed)
.. ❌ Not Supported ❌
.. error: NoSuchAlgorithmException
Trying ML-DSA-65
.. initialized.
PQC Checker
Filter Apks
PQC Checker
.. package: com.yubico
.. signed with EC, EC,
.. ✅ Pqc signed ✅
Play Console
.. package:
com.google.android.app
.. signed with RSA, RS
.. ✅ Pqc signed ✅
.. signed with RSA
.. ❌ Not pqc signed ❌
Tethering
.. package:
com.google.android.networkstack.tethering
.. signed with RSA
radar.cloudflare.com/po
RADAR
ryption
t-quantum
necting using the
key agreement,
um secure.
xchange algorithm
ser connection to
sers like Chrome
x 132+ support
key exchange
Hostname
google.com
Authenticator selection
Other options
challenge
d594bfa863ac07f78b20a9f65d88
timeout (milliseconds)
90000
pubKeyCredParams
ML-DSA-44
ML-DSA-65
ML-DSA-87
ES256
ES384
ES512
EdDSA
RS1
RS256
hybrid
security-key
EDIT
CREATE
Interact
Session
github.com/YubicoLabs/pqc-checker-android

Further Reading



github.com/YubicoLabs/pqc-checker-android

FIPS-203: ML-KEM (Algorithm specification)

FIPS-204: ML-DSA (Algorithm specification)

FIPS-205: SLH-DSA (Algorithm specification)

Shor's Algorithm (Factorization)

AI Million-Dollar Prize Math Problem (Navier-Stokes fluids solved by AI controversy)

Cryptographically Weakness in NIST PQC (Anthropic and NIST not continuing with HAWK)

NIST Post Quantum Cryptography (Americans)

BUND Post Quantum Recommendations (Germans)

Bouncy Castle (3rd party crypto support)

BOTAN (© native implementation)

Signing Android APKs with apksigner (cillyvms 🫶, whoever you are)